

Optimization

3.2.1 Random search

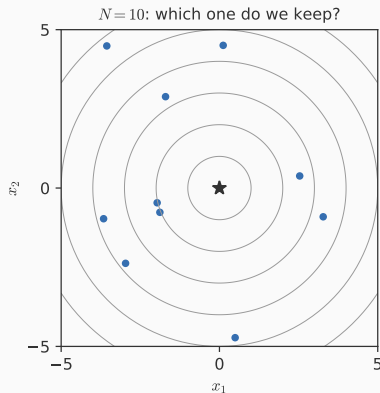
José Firmino Aguilar Madeira

No derivatives and no direction: which point do we keep?

3.1 ended with: *if we do not know ∇f , how do we choose where to move?* First answer: **we do not choose** — we try random points.

$f(x_1, x_2) = x_1^2 + x_2^2$ on $[-5, 5]^2$; we generate 10 uniform points.

Which of these points would we keep?



Accumulated samples: the best so far never gets worse.

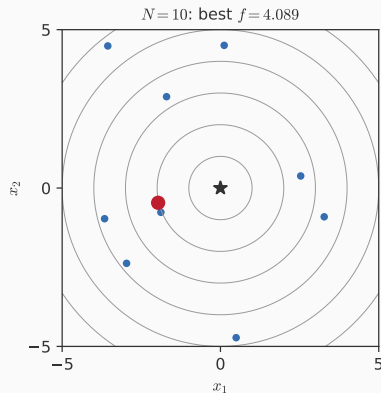
No derivatives and no direction: which point do we keep?

3.1 ended with: *if we do not know ∇f , how do we choose where to move?* First answer: **we do not choose** — we try random points.

$f(x_1, x_2) = x_1^2 + x_2^2$ on $[-5, 5]^2$; we generate 10 uniform points.

Which of these points would we keep?

→ the one with the **smallest f** .



Accumulated samples: the best so far never gets worse.

No derivatives and no direction: which point do we keep?

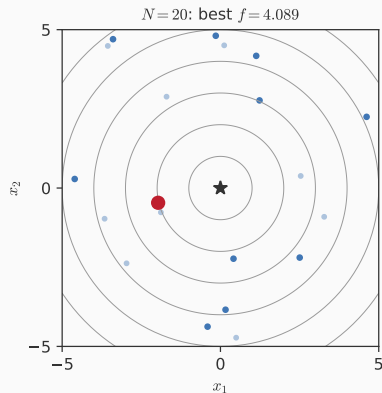
3.1 ended with: *if we do not know ∇f , how do we choose where to move?* First answer: **we do not choose** — we try random points.

$f(x_1, x_2) = x_1^2 + x_2^2$ on $[-5, 5]^2$; we generate 10 uniform points.

Which of these points would we keep?

→ the one with the **smallest f** .

10 more points. And now? We keep the best of *all* — here none of the new ones is better: the same one stays.



Accumulated samples: the best so far never gets worse.

No derivatives and no direction: which point do we keep?

3.1 ended with: *if we do not know ∇f , how do we choose where to move?* First answer: **we do not choose** — we try random points.

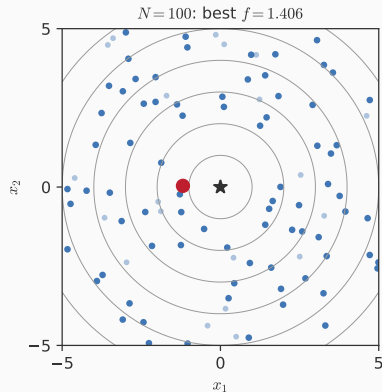
$f(x_1, x_2) = x_1^2 + x_2^2$ on $[-5, 5]^2$; we generate 10 uniform points.

Which of these points would we keep?

→ the one with the **smallest f** .

10 more points. And now? We keep the best of *all* — here none of the new ones is better: the same one stays.

$N = 100$: the best gets closer ($f = 1.406$).



Accumulated samples: the best so far never gets worse.

No derivatives and no direction: which point do we keep?

3.1 ended with: *if we do not know ∇f , how do we choose where to move?* First answer: **we do not choose** — we try random points.

$f(x_1, x_2) = x_1^2 + x_2^2$ on $[-5, 5]^2$; we generate 10 uniform points.

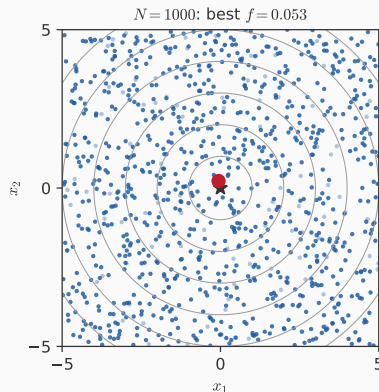
Which of these points would we keep?

→ the one with the **smallest f** .

10 more points. And now? We keep the best of *all* — here none of the new ones is better: the same one stays.

$N = 100$: the best gets closer ($f = 1.406$).

$N = 1000$: $f = 0.053$, close to the minimum.



Accumulated samples: the best so far never gets worse.

No derivatives and no direction: which point do we keep?

3.1 ended with: *if we do not know ∇f , how do we choose where to move?* First answer: **we do not choose** — we try random points.

$f(x_1, x_2) = x_1^2 + x_2^2$ on $[-5, 5]^2$; we generate 10 uniform points.

Which of these points would we keep?

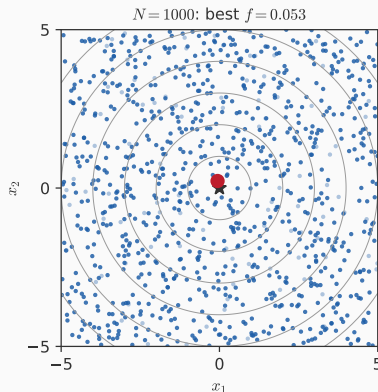
→ the one with the **smallest f** .

10 more points. And now? We keep the best of *all* — here none of the new ones is better: the same one stays.

$N = 100$: the best gets closer ($f = 1.406$).

$N = 1000$: $f = 0.053$, close to the minimum.

generate → evaluate → keep the best



Accumulated samples: the best so far never gets worse.

So the algorithm is almost trivial

Pure random search: domain $D = [a, b]$, N samples

```
1:  $f_{\text{best}} \leftarrow +\infty$ 
2: for  $i = 1, \dots, N$  do
3:   generate  $\mathbf{x}_i$  uniform in  $D$ 
4:   compute  $f(\mathbf{x}_i)$ 
5:   if  $f(\mathbf{x}_i) < f_{\text{best}}$  then  $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}_i$ ;  $f_{\text{best}} \leftarrow f(\mathbf{x}_i)$ 
6:   end if
7: end for
8: return  $\mathbf{x}_{\text{best}}$ 
```

$$n_f = N$$

- No direction, no step, no memory: the points are independent (and therefore parallelizable).
- Uses only values of f (order 0) and a bounded domain D .
- Guarantee: under the usual assumptions (compact domain, sampling with full support, f continuous), the best value approaches the *global* optimum in probability as $N \rightarrow \infty$ — slowly.

How slowly? That is the next question.

Experimental analysis: how fast does it get closer?

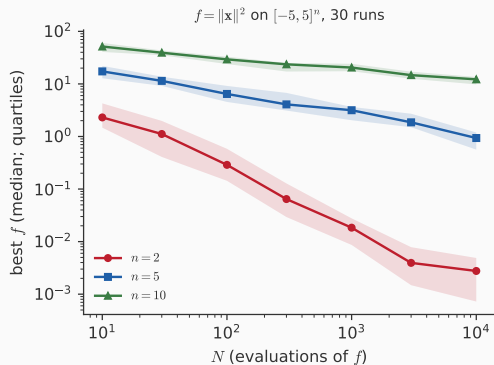
$f = \|\mathbf{x}\|^2$ on $[-5, 5]^2$, 30 runs (stochastic method: median and quartiles):

N	best f	$\ \mathbf{x}_{\text{best}}\ $ [Q1, Q3]
10	2.3	1.5 [1.2, 2.1]
100	0.29	0.54 [0.38, 0.76]
1 000	0.018	0.14 [0.09, 0.17]
10 000	0.0028	0.053 [0.027, 0.070]

For $n = 2$, multiplying N by 10 typically divides the distance by $\sqrt{10} \approx 3$. In general:

$\|\mathbf{x}_{\text{best}} - \mathbf{x}^*\| \sim N^{-1/n}$ (here $f = \|\mathbf{x}\|^2$, so $f_{\text{best}} \sim N^{-2/n}$). For $\|\mathbf{x}\| \approx 10^{-3}$: $N \sim 10^7$.

In this separable problem, an efficient one-dimensional search in each coordinate reaches that accuracy with a few dozen evaluations.



Line: median; band: quartiles. The larger n , the slower.

The curse of dimensionality

Why does the distance decrease like $N^{-1/n}$? The probability that a uniform point in $[-5, 5]^n$ falls in the ball of radius ε around the minimum is

$$p(\varepsilon) = \frac{\text{vol}(B_\varepsilon)}{10^n} \propto \left(\frac{\varepsilon}{10}\right)^n,$$

so we need $N \sim 1/p \propto (10/\varepsilon)^n$ samples to expect one hit. Ignoring the geometric constant $\text{vol}(B_1)$, it is the **scale** that matters, not the exact number.

Question: if, ignoring the constant, in 2D the scale is 10^4 (with $\varepsilon = 0.1$), what happens in 10D?

The curse of dimensionality

Why does the distance decrease like $N^{-1/n}$? The probability that a uniform point in $[-5, 5]^n$ falls in the ball of radius ε around the minimum is

$$p(\varepsilon) = \frac{\text{vol}(B_\varepsilon)}{10^n} \propto \left(\frac{\varepsilon}{10}\right)^n,$$

so we need $N \sim 1/p \propto (10/\varepsilon)^n$ samples to expect one hit. Ignoring the geometric constant $\text{vol}(B_1)$, it is the **scale** that matters, not the exact number.

Question: if, ignoring the constant, in 2D the scale is 10^4 (with $\varepsilon = 0.1$), what happens in 10D?

	$n = 2$	$n = 5$	$n = 10$
scale $(10/\varepsilon)^n, \varepsilon = 0.1$	10^4	10^{10}	10^{20}

With the constant: $1/p \approx 3 \cdot 10^3, 2 \cdot 10^9, 4 \cdot 10^{19}$ — the numbers change, the scale does not.

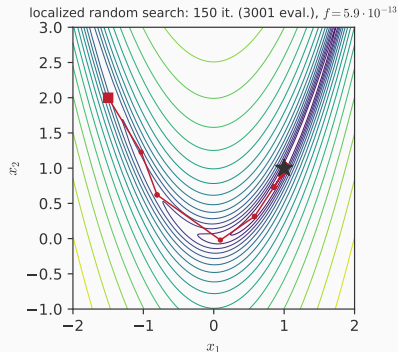
The previous figure shows the same: for $n = 10$, 10^4 evaluations still give $f \approx 12$ (median). Pure random search is only viable in very low dimension or as the **first phase** of another method.

Why not concentrate the search? The localized variant

If always sampling the whole domain is so inefficient, we sample **around the best point**, in $\mathbf{x} + r[-1, 1]^n$, and shrink r when there is no progress:

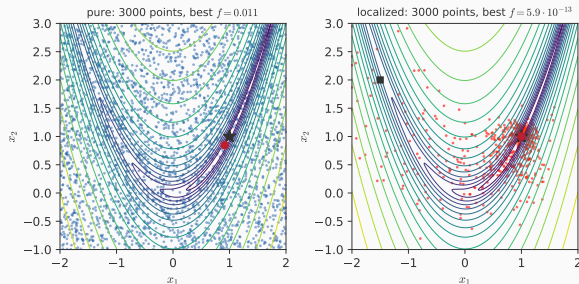
```
1:  $\mathbf{x} \leftarrow \mathbf{x}_0$ ;  $r \leftarrow r_0$ ;  $k \leftarrow 0$ 
2: loop
3:   generate  $m$  points  $\mathbf{x} + \xi$ ,  $\xi$  uniform in  $[-r, r]^n$ 
4:   if the best of them improves  $f(\mathbf{x})$  then accept:
      $\mathbf{x} \leftarrow$  that point
5:   else  $r \leftarrow \gamma r$  ( $\gamma \approx 0.9$ )
6:   end if
7:    $k \leftarrow k + 1$ ; stop if  $r \leq \epsilon_x$  or  $k \geq k_{\max}$ 
8: end loop
```

It gains **localization**: it explores around the best point $\mathbf{x}$, with a search scale r . It loses the global property and becomes a local search: it may approach a local minimum, but the result depends on the starting point, the samples and the parameters.



Rosenbrock from $(-1.5, 2)$, $m = 20$, $r_0 = 1$, $\gamma = 0.9$: one run (150 iterations, $n_f = 3001$, with $f(\mathbf{x}_0)$) reaches the valley and follows it to $(1, 1)$. In 30 runs (seeds 0–29), it reaches $f < 10^{-4}$ with $n_f = 975$ [867, 1052] (median [Q1, Q3]).

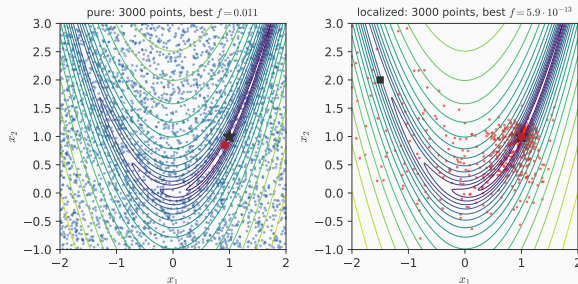
Pure or localized? The same budget on Rosenbrock



One run each, $n_f = 3000$ (pure) and 3001 (localized, with $f(\mathbf{x}_0)$); on the right, colour shows the order of the samples (red: first).

On this problem, with these parameters.

Pure or localized? The same budget on Rosenbrock



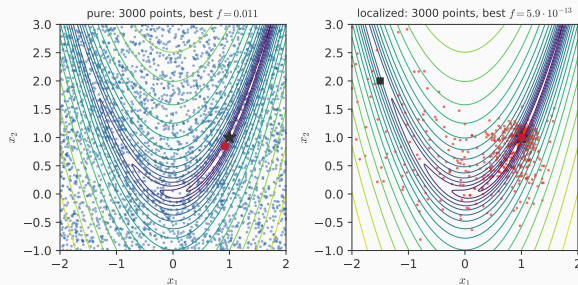
Which uses the evaluations better?

The localized one: it focuses and follows the valley. In 30 runs, best f , median [Q1, Q3]: 0.014 [0.008, 0.022] (pure) vs. $2 \cdot 10^{-12}$ [$5 \cdot 10^{-13}$, $8 \cdot 10^{-12}$] (localized).

One run each, $n_f = 3000$ (pure) and 3001 (localized, with $f(\mathbf{x}_0)$); on the right, colour shows the order of the samples (red: first).

On this problem, with these parameters.

Pure or localized? The same budget on Rosenbrock



One run each, $n_f = 3000$ (pure) and 3001 (localized, with $f(\mathbf{x}_0)$); on the right, colour shows the order of the samples (red: first).

On this problem, with these parameters.

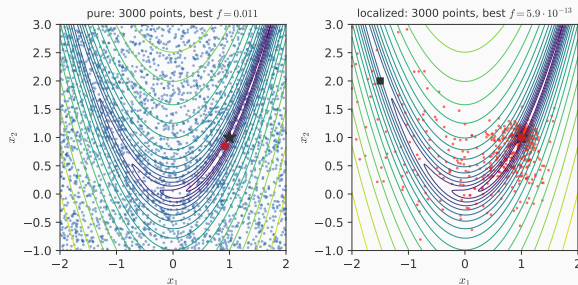
Which uses the evaluations better?

The localized one: it focuses and follows the valley. In 30 runs, best f , median [Q1, Q3]: 0.014 [0.008, 0.022] (pure) vs. $2 \cdot 10^{-12}$ [$5 \cdot 10^{-13}$, $8 \cdot 10^{-12}$] (localized).

And which can escape to another region?

The pure one: it covers the whole domain; the localized one sees only its own neighbourhood.

Pure or localized? The same budget on Rosenbrock



One run each, $n_f = 3000$ (pure) and 3001 (localized, with $f(\mathbf{x}_0)$); on the right, colour shows the order of the samples (red: first).

On this problem, with these parameters.

Which uses the evaluations better?

The localized one: it focuses and follows the valley. In 30 runs, best f , median [Q1, Q3]: 0.014 [0.008, 0.022] (pure) vs. $2 \cdot 10^{-12}$ [$5 \cdot 10^{-13}$, $8 \cdot 10^{-12}$] (localized).

And which can escape to another region?

The pure one: it covers the whole domain; the localized one sees only its own neighbourhood.

global exploration



efficient local exploration

A trade-off that reappears in Ch. 5.

On the computer: the whole algorithm

```
rng(s);                % seed
fbest = inf;
for i = 1:N
    x = a + (b-a).*rand(1,n); % in [a,b]
    fx = f(x);
    if fx < fbest
        xbest = x;
        fbest = fx;
    end
end
```

$$n_f = N$$

The full functions (RandomSearch, LocalRandomSearch, with info.nfev and info.nhit, the cost to reach the target used in the final comparison of the chapter) are in the course companion code.

Stochastic algorithm $\Rightarrow$ several runs

rng(s) fixes the seed: each run is reproducible, but the result is a **random variable**. We report the **median and quartiles** of 20–30 runs (different seeds), never a single run.

This rule returns in simulated annealing (5.2), genetic algorithms (5.3) and NSGA-II (6.6).

KEY TAKEAWAYS — 3.2.1

1. Pure random search: **generate** → **evaluate** → **keep the best**.
2. Uses only values of f (**order 0**) and costs $n_f = N$ evaluations.
3. With uniform sampling, $\|\mathbf{x}_{\text{best}} - \mathbf{x}^*\| \sim N^{-1/n}$: it suffers strongly from the **dimension**.
4. Localizing the search improves local efficiency, but **loses the global nature** and makes it depend on the starting point, the samples and the parameters.
5. It is **stochastic**: a single run is not assessed on its own — several seeds, median and quartiles.

A derivative-free method is only interesting if it uses the evaluations **better than random points** — with the same n_f , on the same problem.

Next (3.2.2): we scattered points **at random**. What if we want to cover the domain **systematically**? — grid search.

References Full list: *Course bibliography*, at the end of the course PDF.

- Conn, A. R., Scheinberg, K., Vicente, L. N., *Introduction to Derivative-Free Optimization*, SIAM (MPS-SIAM Series on Optimization 8), 2009 — Ch. 1.
- Arora, J. S., *Introduction to Optimum Design*, 3rd ed., Academic Press, 2011 — Ch. 18.

Resources

- Companion code: `RandomSearch`, `LocalRandomSearch` (`info.nfev`, `info.nhit`). MATLAB: `rng(s)`.